

1. What is property-service?

Property Service manages the actual Restaurant or KTV business/location that operates on our platform.

Our services answer different business questions:

user-service

"Who is this person?"

property-owner-service

"Which organization/business owner does this person represent?"

access-control-service

"What is this person allowed to do?"

property-service

"Which Restaurant/KTV locations exist and what is their lifecycle?"

For example:

User

Piseth

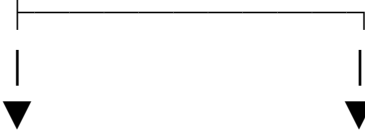
ID: U001



Property Owner / Organization

Piseth Group

ID: O001



Property P001

Angkor Restaurant

RESTAURANT

Property P002

Family KTV

KTV

2. User, Owner and Property are NOT the same thing

This is especially important.

user-service

Represents the **person/account**.

User

id = U001

name = Piseth

email = ...

keycloakUserId = ...

property-owner-service

Represents the **business operator / organization**.

PropertyOwner

id = O001

userId = U001

ownerType = COMPANY

businessName = Piseth Group

verificationStatus = VERIFIED

active = true

property-service

Represents the actual business locations.

Property

id = P001

ownerId = O001

name = Angkor Restaurant
businessType = RESTAURANT
status = APPROVED

One owner can have many properties:

U001

|



O001 Piseth Group

|

- P001 Angkor Restaurant
- P002 Siem Reap Restaurant
- P003 Family KTV
- P004 Angkor KTV

Therefore:

Property.ownerId

references the **organization/property owner**, not the user.

3. organizationId vs propertyId

This is now very important because of Access Control.

In this architecture:

organizationId = property_owner.id

For example:

organizationId = O001

All businesses belonging to that owner sit underneath it:

Organization O001

|

- Restaurant P001
- Restaurant P002
- KTV P003

Therefore an owner role can be:

userId = U001

```
role      = RESTAURANT_OWNER
scopeType = ORGANIZATION
scopeId   = 0001
```

But a Restaurant Manager can later be:

```
userId    = U010
role      = RESTAURANT_MANAGER
scopeType = RESTAURANT
scopeId   = P001
```

This is why we must **not treat organizationId and propertyId as the same thing**.

4. Why do Restaurant and KTV belong in the same Property Service?

Because they share the same core business identity:

Restaurant / KTV

- |
- Owner
- Name
- Description
- Address
- Location
- Phone
- Email
- Telegram
- Facebook
- Images
- Status
- Active

We distinguish them using:

```
public enum BusinessType {
    RESTAURANT,
    KTV
}
```

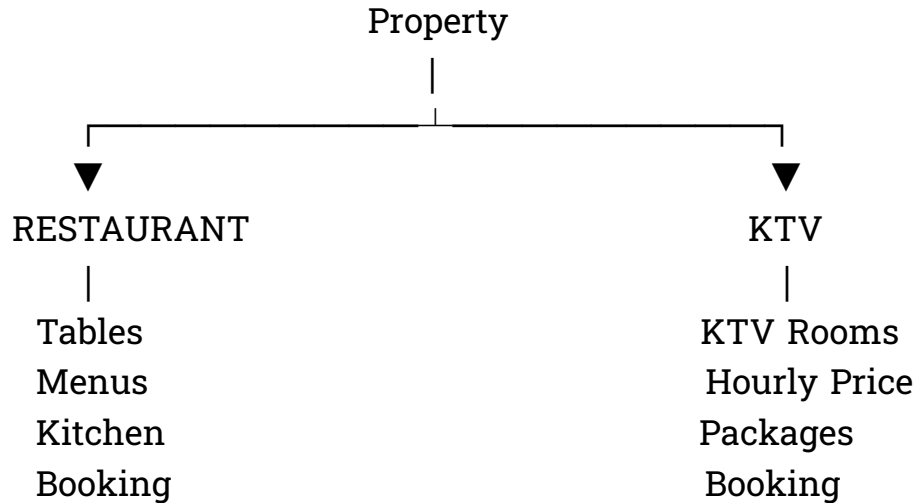
This avoids duplicating:

restaurant-service

ktv-service

when most of the basic business profile is identical.

Later their specialized domains can differ:



property-service owns the **common business identity**, not every Restaurant/KTV feature.

5. Property lifecycle

We should not think:

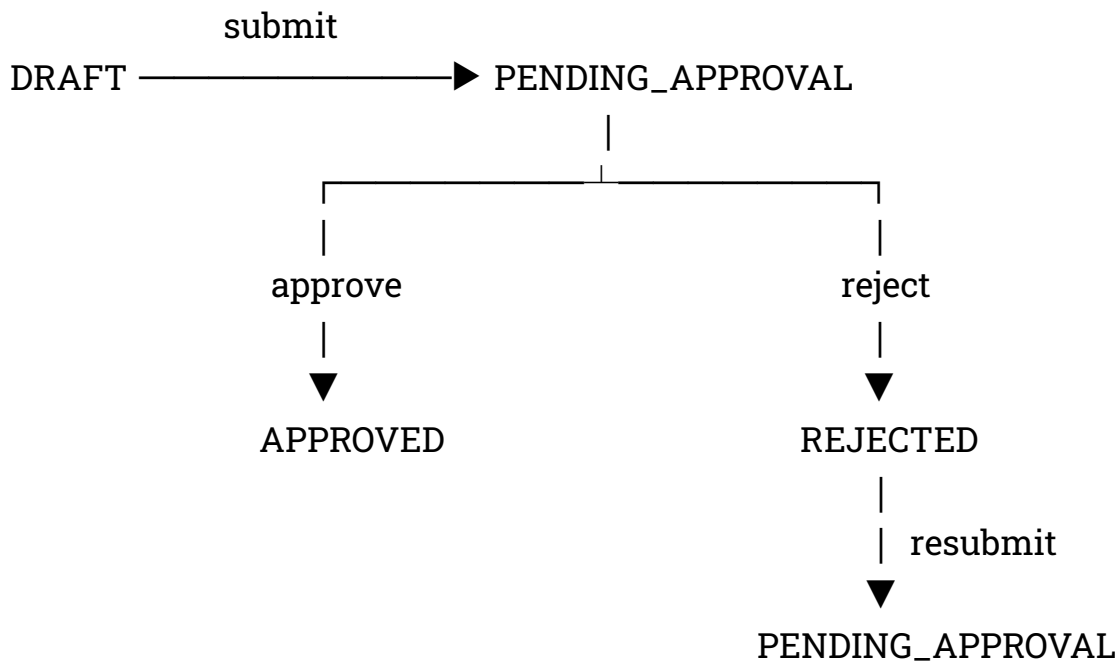
Property Service = CRUD.

It contains real business rules.

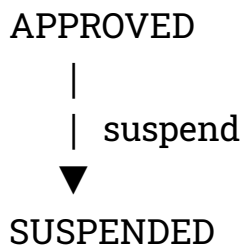
A new property starts:

DRAFT

Then:



An approved property can also become:



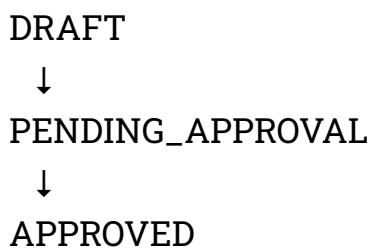
This is effectively a **state machine**.

The backend protects those transitions.

For example:

DRAFT → APPROVED
should not be allowed directly.

The correct flow is:



6. Why backend validation matters

Suppose Angular hides the Approve button.

That does **not** protect the system.

Someone can call:

Postman

curl

Mobile App

Another frontend
directly.

Therefore:

Frontend validation



Good User Experience

Backend validation

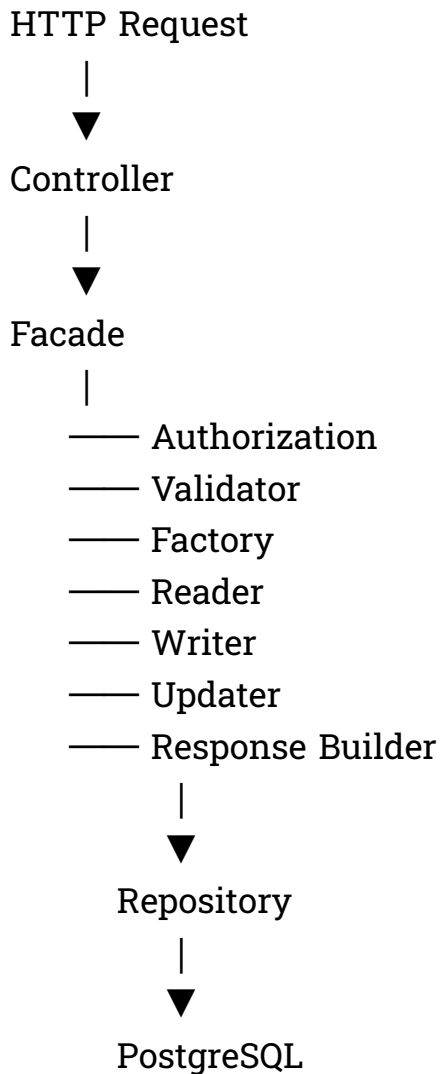


Business Protection

The backend is the authority for business rules.

7. Architecture inside Property Service

The code follows this structure:



Each part has a responsibility.

Controller

Handles HTTP:

Request

Validation

Path variables

Query parameters

Response status

It should not contain core business logic.

Facade

Represents a **use case**.

For example:

Create Property

Update Property

Submit Property

Approve Property

Search Property

Facade orchestrates several components.

Validator

Protects business rules.

Factory

Knows how to construct a valid new Property.

For example:

new property

↓

status = DRAFT

active = true

ownerId = organizationId

businessType = requested type

Reader / Writer

Provide domain-oriented persistence operations.

Repository

Deals with PostgreSQL/R2DBC.

This is much better than:

Controller

↓

One giant Service

↓

Repository

where eventually one Service class contains hundreds or thousands of lines.

8. The important integration change

Before integration, our request contained:

```
{  
  "ownerId": "...",  
  "name": "Piseth Restaurant",  
  "businessType": "RESTAURANT"  
}
```

That was acceptable for the standalone testing phase.

But it is **not appropriate for the integrated system**.

Imagine I'm logged in as:

User U001

Owner 0001

and I send:

```
{  
  "ownerId": "O999"  
}
```

I could attempt to create a Restaurant under another organization.

So we removed ownerId from CreatePropertyRequest.

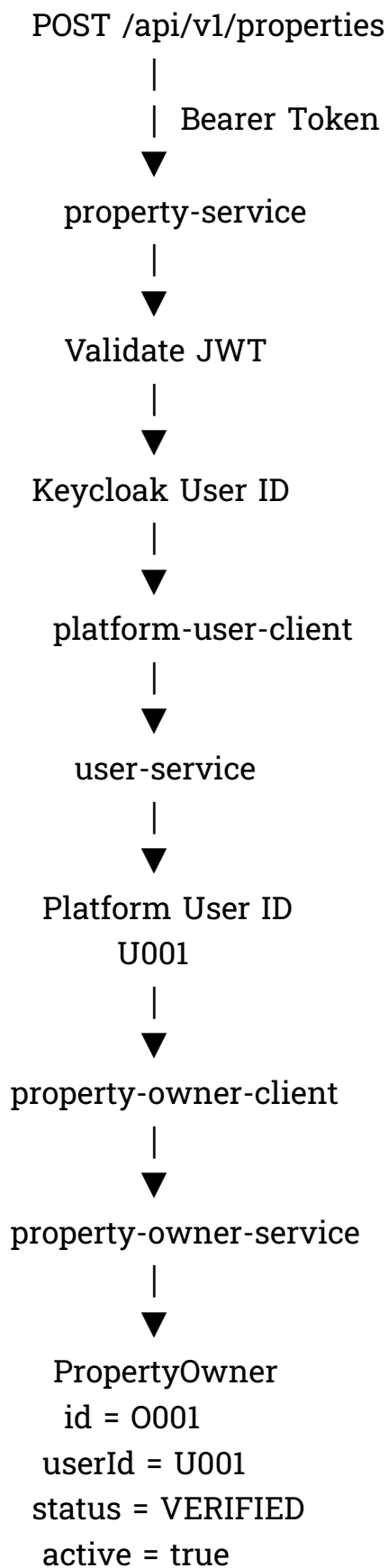
The client now sends:

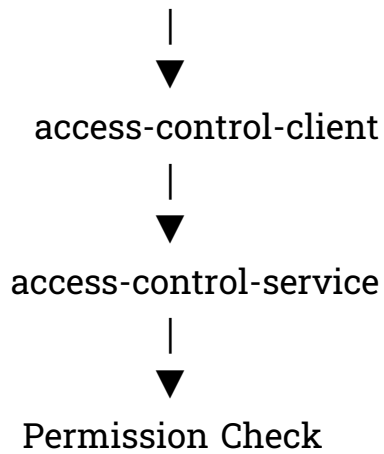
```
{  
  "name": "Piseth Restaurant",  
  "businessType": "RESTAURANT",  
  "description": "...",  
  "provinceCode": "12",  
  "districtCode": "1201"  
}
```

The backend determines ownership.

That is an important security improvement.

9. New Create Property flow





Then business type determines the required permission.

For Restaurant:

RESTAURANT_CREATE

For KTV:

KTV_CREATE

The permission is checked at:

scopeType = ORGANIZATION

scopeId = 0001

If authorized:

Create Property



ownerId = 0001



status = DRAFT



active = true



Save PostgreSQL

10. Complete Restaurant example

Suppose Piseth logs in.

Keycloak token contains:

sub = K001

That is a **Keycloak ID**.

It is not our application's user ID.

So:

K001

|

| platform-user-client



user-service

|



U001

Now Property Service knows:

current platform user = U001

Next:

U001

|

| platform-property-owner-client



property-owner-service

|



O001

Property Service verifies:

verificationStatus = VERIFIED

active = true

Then:

U001 + O001 + RESTAURANT_CREATE



access-control-service

Access Control might have:

UserRole

userId = U001

role = RESTAURANT_OWNER

scopeType = ORGANIZATION

scopeId = 0001

and:

RESTAURANT_OWNER



RESTAURANT_CREATE

Therefore:

allowed = true

Property Service finally creates:

Property

id = P001

ownerId = 0001

name = Piseth Restaurant

businessType = RESTAURANT

status = DRAFT

active = true

Notice the separation:

K001 = Keycloak identity

U001 = Platform user

O001 = Organization / Property Owner

P001 = Restaurant

11. Why Property Owner must be VERIFIED

We added another business protection to the integrated flow.

Creating properties requires:

PropertyOwner
active = true

AND

verificationStatus = VERIFIED

So this won't work:

Owner
status = DRAFT

↓

Create Restaurant ❌

Neither will:

Owner
status = SUSPENDED

↓

Create Restaurant ❌

The normal business flow becomes:

Register Business Owner

↓

Property Owner created

↓

DRAFT

↓

Submit verification

↓

PENDING_REVIEW

↓

Platform verifies



VERIFIED



Create Restaurant / KTV

That connects our services through an actual business process rather than integrating them simply because they are microservices.

12. Authentication vs Authorization

This service is also a good place to learn this distinction.

Authentication

Who are you?

Handled primarily by:

Keycloak

JWT

platform-security

user-service identity mapping

Authorization

Are you allowed to create this Restaurant?

Handled by:

access-control-service

Example:

Authenticated?

YES



U001



Has RESTAURANT_CREATE
for organization 0001?



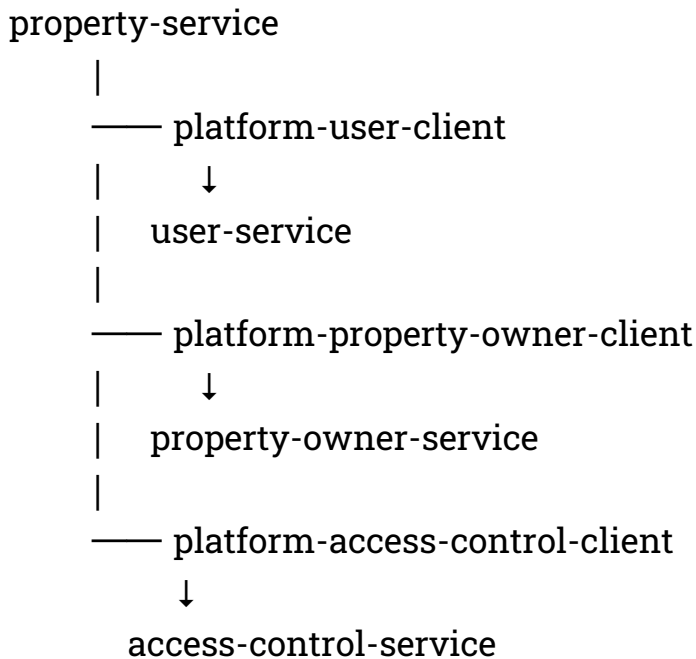
YES/NO

Being logged in does **not** automatically mean being authorized.

13. Why use Platform Clients?

Property Service does not implement raw WebClient code everywhere.

Instead:



This keeps service-to-service communication contracts reusable.

For example:

```
propertyOwnerClient.getByUserId(userId)
```

is much clearer inside business code than repeatedly writing:

```
webClient.get()
    .uri(...)
    .header(...)
    .retrieve()
    ...
```

Platform library:

How do I communicate with Property Owner Service?

Property Service:

Why do I need Property Owner Service for this use case?

That separation is important.

14. What changed in the code

We updated platform-property-owner-client with an owner lookup contract:

```
Mono<PropertyOwnerSummary> getByUserId(UUID userId);
```

and added the summary DTO needed by consumers.

We extended platform-access-control-client with permission checking:

```
Mono<Boolean> hasPermission(  
    UUID userId,  
    String permissionCode,  
    AccessScopeType scopeType,  
    UUID scopeId  
);
```

It uses our existing Access Control endpoint:

```
POST /api/v1/access-control/check-permission
```

We also added:

```
AccessScopeType  
PermissionCheckRequest  
PermissionCheckResponse  
and bearer-token propagation for authenticated service calls.
```

15. New Property Service components

We deliberately avoided putting everything into CreatePropertyFacadeImpl.

We added:

```
CurrentPropertyOwnerResolver  
CurrentPropertyOwnerResolverImpl  
Responsibility:
```

```
Current User
```

↓

Find Property Owner



Check VERIFIED



Check active



Return organization

And:

PropertyAuthorizationService

PropertyAuthorizationServiceImpl

Responsibility:

BusinessType



Required permission



Access Control



Allow / Deny

For example:

```
switch (businessType) {  
    case RESTAURANT -> "RESTAURANT_CREATE";  
    case KTV -> "KTV_CREATE";  
}
```

This keeps the Facade focused on orchestration and follows SRP.

16. CreatePropertyFacade is now an orchestrator

Conceptually, it now does:

resolve owner



authorize



validate property data
↓
factory.createDraft(...)
↓
save property
↓
save image relationships
↓
build response

It coordinates the use case without becoming the implementation of every responsibility.

17. Auditing

The integration also activates our existing auditing architecture.

JWT
|
▼
Keycloak user ID
|
▼
platform-user-client
|
▼
user-service
|
▼
Platform user ID
|
▼
SecurityCurrentAuditorProvider
|
▼
createdBy / updatedBy

Therefore if:

U001

creates:

P001

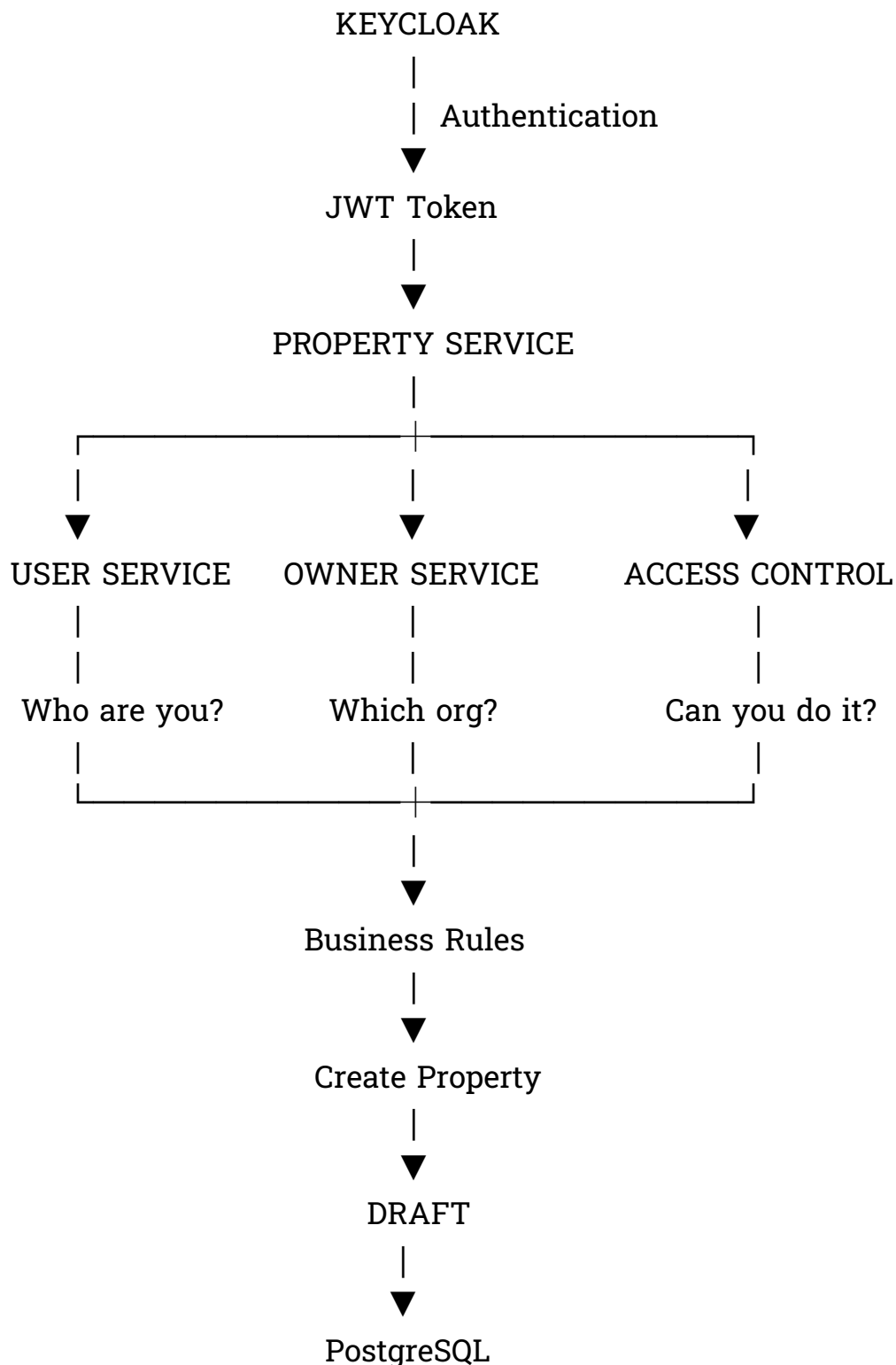
the property can have:

createdBy = U001

rather than storing the Keycloak UUID.

That's the correct separation between external identity provider identity and our platform's business identity.

18. What students should remember



There are five different concepts here:

Authentication

≠

User Identity

≠

Business Ownership

≠

Authorization

≠

Business Logic

A lot of junior code mixes all five together.

Our architecture gives us a very good real example for learning why they should be separated.